# HackDNA – Secrets in Source

## Challenge Overview

This challenge demonstrates how sensitive information can accidentally be exposed within the source code of a web application. Attackers frequently inspect frontend assets during reconnaissance to discover hidden credentials, internal comments, API keys, and development artifacts.

Source-code disclosure vulnerabilities are among the most common security weaknesses found in modern web applications.

---

# Objective

Inspect the application's source code and identify hidden secrets or sensitive information exposed to the client.

---

# Understanding Source Code Exposure

Everything delivered to the browser should be considered public.

Attackers routinely analyze:

- HTML source
- JavaScript files
- CSS comments
- Hidden form fields
- API requests
- Client-side configuration files

Developers sometimes unintentionally expose:

- Passwords
- API keys
- Debug comments
- Internal endpoints
- Tokens
- Administrative functionality

---

# Reconnaissance

Open the target webpage and inspect the source code.

View source using:

```
CTRL + U
```

or:

```
Right Click → View Page Source
```

Review the entire HTML document carefully.

---

# Initial Discovery

Sensitive information may appear inside comments.

Example:

```
<!-- Temporary admin password: admin123 -->
```

or:

```
<!-- TODO: remove debug credentials before deployment -->
```

Hidden fields may also reveal sensitive values.

Example:

```
<input type="hidden" value="administrator">
```

---

# JavaScript Analysis

Applications often expose logic inside JavaScript files.

Inspect loaded scripts:

```
<script src="main.js"></script>
```

Use browser developer tools:

```
F12 → Sources
```

Search for sensitive keywords:

```
password
secret
admin
token
apikey
internal
```

---

# Example Vulnerable Code

Hardcoded credential:

```
const adminPassword = "SuperSecret123";
```

Exposed API token:

```
const api_key = "dev-api-key-001";
```

Debug endpoint:

```
const debug_url = "/admin/debug";
```

Attackers use these discoveries during exploitation.

---

# Exploitation

Discovered credentials or hidden endpoints may provide:

- Administrative access
- API interaction
- Authentication bypass
- Hidden functionality
- Privilege escalation

Example workflow:

1. Discover hidden admin credential in source
2. Navigate to login page
3. Authenticate using exposed password
4. Gain unauthorized access

---

# Root Cause

The vulnerability exists because sensitive operational data was embedded directly into client-side resources.

Common causes include:

- Poor development practices
- Debugging leftovers
- Hardcoded secrets
- Incomplete deployment sanitization
- Misconfigured frontend applications

---

# Security Impact

Source-code disclosure may lead to:

- Credential compromise
- Account takeover
- Internal infrastructure exposure
- API abuse
- Privilege escalation
- Full application compromise

Attackers heavily rely on exposed information during reconnaissance.

---

# Detection Techniques

## Manual Inspection

- View source code
- Review comments
- Analyze hidden fields
- Inspect JavaScript files
- Monitor network requests

## Automated Secret Scanning

Using grep:

```
grep -Ri "password\|secret\|apikey\|token" .
```

Using Git tools:

```
gitleaks detect
```

Using browser DevTools:

```
F12 → Network → JS Files
```

---

# Defensive Measures

## Never Store Secrets Client-Side

Sensitive information must remain server-side.

## Remove Debug Information

Eliminate comments and development artifacts before deployment.

## Use Environment Variables

Secrets should be managed using:

- Environment variables

- Secret managers
- Vault solutions

### Conduct Secure Code Reviews

Implement:

- Static analysis
- Secret scanning
- CI/CD security checks
- Manual audits

### Apply Principle of Least Privilege

Even leaked tokens should have restricted permissions.

---

## Secure Development Example

Instead of:

```
const db_password = "root123";
```

Use secure backend authentication with protected environment variables.

---

## Real-World Examples

Numerous real-world incidents involve exposed secrets in:

- GitHub repositories
- Frontend JavaScript
- Mobile applications
- Backup files
- CI/CD pipelines

Frequently leaked secrets include:

- AWS keys
- Database passwords
- OAuth tokens
- SMTP credentials
- Firebase configurations
- JWT secrets

## Key Takeaway

If sensitive information is accessible in the browser, attackers can retrieve it.

Frontend code must never contain operational secrets or authorization logic.

---

## Vulnerability Classification

- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
- CWE-798: Use of Hard-coded Credentials
- CWE-215: Information Exposure Through Debug Information
- OWASP A05:2021 – Security Misconfiguration

---

## Conclusion

Source-code inspection is one of the first reconnaissance techniques used during penetration testing. Exposed secrets significantly increase organizational risk and often enable rapid exploitation.

Secure development practices, automated scanning, and strict secret management policies are essential to prevent information disclosure vulnerabilities.